

EXERCISE 2 · AFTER DECK 1 · ANSWER KEY

Tokens and Multi-Turn Conversations

Pressure-test the stateless-model mental model — tokens, context, history, cost.

6 questions · For trainer use · Distribute after the exercise

HOW TO USE

Walk through each answer with the cohort. The explanation is what matters — many of these have answers that look right but fail for subtle reasons.

Q1 OF 6 · CONCEPT

True or false: when you use a chat interface like ChatGPT or Claude.ai, the model **remembers** your previous messages across turns.

- A. True — the model has a built-in memory module that persists conversation state.
- B. False — the model is stateless; the UI re-sends the full conversation on every call.
- C. True for paid plans, false for free plans.
- D. True only within the same 24-hour window.

ANSWER

B.

Models are stateless at the API level. "Memory" is illusory — the application layer (the chat UI) is collecting the conversation history and submitting all of it on every call. There is no server-side memory of your previous turns at the model level.

Q2 OF 6 · BUG SPOT

A learner tries to do multi-turn conversation. Their code below produces nonsense on turn 2 — the model doesn't seem to remember turn 1. Spot the bug.

```
conversation_history = []

# Turn 1
response_1 = client.converse(
    modelId=model_id,
    messages=[{"role": "user", "content": [{"text": "I like sci-fi."}]}],
    inferenceConfig={"maxTokens": 200},
)

# Turn 2
response_2 = client.converse(
    modelId=model_id,
    messages=[{"role": "user", "content": [{"text": "Recommend something."}]}],
    inferenceConfig={"maxTokens": 200},
)
```

ANSWER

Neither turn appends to `conversation_history`, and turn 2 doesn't include turn 1 in `messages`.

The variable `conversation_history` is declared but never used. Each `converse()` call sends a fresh single-message list, so the model sees turn 2 as a brand-new conversation with no context. Fix: after each call, append the user message and the assistant message to history, and pass `messages=conversation_history` in every subsequent call.

Q3 OF 6 · CONCEPT

A model's context window is 200,000 tokens. Your system prompt is 500 tokens. After 30 turns, your conversation history has grown to 198,000 tokens. What happens on the next call?

- A. The model summarizes earlier turns automatically to free up space.
- B. The model truncates the oldest messages and continues silently.
- C. The API returns a `ValidationException` because input plus expected output exceeds the window.
- D. The model responds normally but charges you double tokens.

ANSWER

C.

Bedrock does not auto-summarize or auto-truncate. If your input plus reserved output space exceeds the context window, the API returns an error. Your code is responsible for managing history length — cap it, summarize old turns, or persist + reload. Production agents always need a strategy here.

Q4 OF 6 · CONCEPT

Why is `maxTokens` described in the deck as your "hard ceiling against a runaway bill"?

- A. It caps how many input tokens the model can read.
- B. It caps how many output tokens the model can generate.
- C. It caps how many API calls you can make per minute.
- D. It caps how many models you can use in a single account.

ANSWER

B.

`maxTokens` bounds the output. Output tokens are typically 3-5× more expensive than input tokens, and a model that's asked an ambiguous question can sometimes generate thousands of tokens of unnecessary explanation. `maxTokens` is the only hard control you have to stop that.

Q5 OF 6 · BUG SPOT

Spot the conceptual error in this estimate:

"My system prompt is 1,000 tokens. I'm planning 20 turns. So the total tokens I'll send is $1,000 + 20 = 1,020$ tokens. Easy."

ANSWER

The estimate ignores that the system prompt and the full conversation history are sent *on every turn*.

Per turn, you send: system prompt + entire prior conversation + new user message. By turn 20, you're paying for the system prompt 20 times, turn 1's content 19 more times, and so on. Actual input tokens grow roughly quadratically with conversation length. This is why prompt caching, history capping, or summarization matter in production.

Q6 OF 6 · DESIGN SCENARIO

You're building a customer-facing chatbot. Conversations sometimes go past 50 turns. List **two different strategies** to keep cost and context-window usage under control, and name one tradeoff each strategy introduces.

ANSWER

Any two of: cap history length (last N messages), summarize old turns into a rolling summary, persist to a database and load relevant context, use prompt caching for the stable prefix.

Tradeoffs: **Capping** loses old context — the bot may forget facts from turn 3 by turn 50. **Summarization** adds an extra LLM call per turn (or every N turns) and loses fidelity in the compressed summary. **Database persistence** requires you to design retrieval — what to pull back when the user returns. **Prompt caching** saves on the stable prefix but doesn't help with the growing conversation portion. Pick by use case.